

LayerForge: Deterministic Layer Decomposition for AI Output Convergence — An Empirical Report on Newman Modularity Outperforming CPM in Small-N Text Domains

Style: Report framing — not "we claim this" but "we found this to hold". Following a publication philosophy of reader-time-cost > return, we avoid overclaiming and center the paper on reproducible observations. **License:** MIT (same as the rest of the repository). **Reproducibility:** All raw CSV / scripts / JSON outputs for the measurements are bundled under `scripts/k_sweep/`, pinned by commit hash.

Abstract

Conversational AI output tends to include content beyond what the questioner is actually concerned with. We report on **LayerForge**, a deterministic pipeline that decomposes a corpus into 4 ± 1 thematic layers and returns only the layer(s) relevant to a query. The pipeline combines five established components — Cowan's working-memory upper bound, Newman modularity for community detection, SCA-style per-layer distillation, HERCULES-style hierarchical KMeans, and a deterministic core with no LLM in the analysis loop — chosen so that "AI noise filtering" does not itself depend on an AI. We re-implemented a CPM (Constant Potts Model) backend (MIT-pure, self-implemented) to test whether the theoretically resolution-limit-free alternative would outperform Newman in this small-N text domain (operational target $N=20-40$, external scaling check up to $N=100$ on 20 Newsgroups). Across 119-row dual-method correlation sweep, 196-cell $N\times K$ heatmap, 95-row ARI/NMI comparison, and the external 20-Newsgroups benchmark, **the Newman backend exceeded CPM by a large effect size in our test range** (H_{struct} exact match 87.5% vs 19%; 20NG ARI 0.430 vs 0.239 at default $K=3$, Newman reaches 0.557 when $K=4$ is forced; same direction at $N=100$), against the naïve theoretical expectation that CPM should win. **Positioning:** LayerForge shares its core substrate (sentence-embedding similarity + community detection) with prior work in clustering-based topic modeling, notably G2T (Zhang et al. 2023). The contribution of this paper is twofold: (i) the implementation report of combining SCA decomposition, HERCULES hierarchical skeleton, the 4 ± 1 cognitive constraint, and a Mode A/B/C operational layer, and (ii) the systematic empirical comparison of Newman vs CPM in the small-N text regime, including a three-failure-mode mechanism analysis (§5). Concurrently with this work, Bohdal et al. (Samsung 2026) independently arrived at the same problem space on the LLM memory token side; we treat this convergence as evidence that the design space is being identified by multiple independent groups. We report an empirically derived method-selection rule (concept-only; runtime implementation deferred because in the studied domain the rule would output "newman" in essentially all cells). The artifact is published as a Claude Code skill (LayerForge itself does not hold an API key; LLM access at Boundary 1/2 is mediated through Claude Code), and is reproducible end-to-end.

§1. Motivation — Converging by concern

Conversation with AI in natural language has a property that looks "convenient" at first glance but structurally creates a **disadvantage**:

When an AI answers a question, it tends to present related information comprehensively. But the user's "concern" at that moment in the dialogue is **usually just one** (e.g., "I only want to know about the CI/CD of this PR", "I only need to confirm the assumptions of the experimental design").

Comprehensive answers create cost in the following three forms:

1. **Reader cognitive load** The semantic units a human can hold at once are bound by short-term memory capacity. Cowan (2010) estimates the central working-memory capacity at 3-5 chunks. When AI output

exceeds this and includes "out-of-concern topics", the reader pays an **additional cognitive cost to extract the parts that match the concern**.

2. **Bandwidth waste in multi-agent coordination** When AI agent A's output is consumed downstream by AI agent B, comprehensive output from A forces B to either **pay the cost of picking only the concern-relevant region from A's entire output**, or **drift due to being dragged by out-of-concern information**.
3. **Linear waste of AI cost** Processing the full context every turn causes token-priced cost to grow linearly. The essential information per concern is often smaller than the full context, so unnecessary computation is included.

The three are independent phenomena, but **"AI output not converging by concern"** is an **overlapping cause** contributing to all three (token cost also depends on full-context reprocessing, so the source is not perfectly identical, but concern-wise convergence is a partial solution). LayerForge can simultaneously affect all three through convergence of output.

1.1 Research question

Can we reduce the three costs above simultaneously by **converging the AI's output into layers that correspond to each concern**? And can this "layering" be performed **deterministically without re-introducing AI**?

The second question is the important one. **Filtering AI output with AI** produces different outputs for the same input (because of the LLM's stochastic behavior), so the reproducibility of concern-wise convergence is lost. This study **allows AI involvement only at the input boundary, and implements the filtering layer itself deterministically**.

Note that "deterministic" in this paper refers to the **property of the analysis core (detailed in R3)**, and does not include the LLM that may optionally intervene at the input boundary (Boundary 1: natural language $\rightarrow$ structured) or the output boundary (Boundary 2: structured $\rightarrow$ natural language). End-to-end determinism including LLM involvement holds when the Boundaries are switched to mechanical fallback (see §3.1).

1.2 Related work and positioning

LayerForge's design space (sentence embedding + community detection on similarity graph + per-cluster representation + clustering-driven context compression) overlaps with several surrounding research lines, so its position relative to prior art needs to be stated. Below we organize the main literature in four adjacent groups, then state LayerForge's differentiation.

1.2.1 Clustering-based topic modeling and graph-based text clustering A family that clusters document/passage embeddings to extract topic / cluster structure:

- **BERTopic** (Grootendorst 2022, arXiv:2203.05794) — transformer-based document embedding + UMAP dimensionality reduction + HDBSCAN clustering + class-based TF-IDF. Per-document **single-topic** assignment is the default.
- **Top2Vec** (Angelov 2020, arXiv:2008.09470) — Doc2Vec / Sentence Transformer joint embedding + UMAP + HDBSCAN. Topic count is determined automatically.
- **CETopic** (Zhang, Fang, Chen, Namazi-Rad, NAACL 2022, "Is Neural Topic Modelling Better than Clustering?") — SimCSE embedding + UMAP + K-means + TF-IDF $\times$ IDFi variant, with direct comparison against neural topic models.
- **Vec2GC** (Rao & Chakraborty 2021, arXiv:2104.09439) — community detection on weighted similarity graphs over terms / documents; supports density-based and hierarchical clustering.
- **G2T (Graph2Topic)** (Zhang, Liu, Yan 2023, arXiv:2304.06653) — pretrained LM embedding $\rightarrow$ semantic graph construction $\rightarrow$ community detection for topic identification $\rightarrow$ TFIDF variant for word-topic distribution. **The closest prior art to LayerForge's design substrate**. Claims SOTA against BERTopic / CETopic in automatic evaluation.

- **SCA** (Eichin et al. 2024, arXiv:2410.21054) — introduces **per-document multi-topic distribution** on top of clustering, relaxing the single-assignment assumption of plain clustering. The reference implementation for R3 (distillation) in this study.
- **LLM-Assisted Topic Reduction for BERTopic** (Janssens, Bogaert, Van den Poel 2025, arXiv:2509.19365) — a hybrid family that adds an LLM merge step on top of BERTopic output. Differs from the determinism principle (R5) of this study.

These works adopt the task formulation of **”classifying documents/passages into topic groups”**. BERTopic / Top2Vec / CETopic dominantly use density-based (HDBSCAN) or K-means clustering and do not use community detection; Vec2GC / G2T are in the graph-based community-detection family. **G2T is the closest substrate to this work**, but the community-detection algorithm name (Louvain / CPM / Leiden) is not stated at the abstract level, and a systematic Newman-vs-CPM comparison is not performed inside G2T (leaving the originality of §4-§5 of this work intact).

1.2.2 Hierarchical RAG and GraphRAG A family combining hierarchical structure with retrieval / summarization:

- **RAPTOR** (Sarathi et al. 2024, ICLR) — community detection + LLM summarization for hierarchical RAG.
- **GraphRAG** (Edge et al. 2024, arXiv:2404.16130) — Leiden + LLM summarization for knowledge-graph structuring and query-focused summarization.
- **HippoRAG** — PageRank-based retrieval, using graph structure on the index side.
- **Core-based Hierarchies for Efficient GraphRAG** (Hossain & Sarıyüce 2026, arXiv:2603.05207) — replaces GraphRAG's Leiden clustering with **k-core decomposition** to obtain a deterministic hierarchy. The Leiden critique is grounded in ”modularity optimization admits exponentially many near-optimal partitions on sparse graphs”, which is **the same problem attacked from a different angle (replacement)** as the Good (2010) Q-degeneracy we cite in §5. LayerForge stays inside the modularity family and instead documents the failure modes.
- **RAG vs. GraphRAG: A Systematic Evaluation** (Han et al. 2026, arXiv:2502.11371) — proposes a unified evaluation framework for RAG / RAPTOR / HippoRAG2 / GraphRAG. LayerForge sits in the ”passage graph + decoupled summarization” quadrant of this framework.
- **GraphRAG-Bench** (Xiao et al. 2025, arXiv:2506.02404) — a domain-specific reasoning benchmark for the GraphRAG family.
- **HERCULES** (bandeerun/pyhercules, MIT 2025) — a reference implementation of LLM-integrated hierarchical KMeans. No peer-reviewed publication exists at the time of this work; we use it not as a citation base but as a **design skeleton reference** (§2.4).

The bulk of these works assume **LLM integration in the analysis pipeline**. LayerForge, by the determinism principle (R5), **excludes LLM from the analysis core** and restricts LLM involvement to Boundary 1/2 (optional) — an architectural difference. With Core-based GraphRAG we share the problem framing of ”addressing Leiden's reproducibility issue”, but diverge in the solution direction: not replacement (k-core), but **failure-mode documentation + method-selection signal** inside the modularity family.

1.2.3 Context and memory compression A family compressing input under the LLM's context-budget constraint (recent works):

- **Clustering-driven Memory Compression for On-device LLMs** (Bohdal, Saha, Michieli, Ozay, Ceritli 2026, arXiv:2601.17443) — **the closest concurrent prior art to LayerForge in framing**. The mechanism ”groups memories by similarity and merges them within clusters prior to concatenation” is isomorphic to LayerForge's Mode C (multi-agent bandwidth reduction). However, the target is **user-specific memory tokens** (for personalization), and the goal is preserving personalization under a context-budget constraint. LayerForge differs on four axes — (i) **target** (memory tokens vs. corpus passages), (ii) **goal** (personalization vs. concern-wise convergence), (iii) **method** (similarity averaging + merge vs. community detection on similarity graph), (iv) **cognitive constraint** (none vs. Cowan 4 ± 1 explicit). Detailed in §1.2.5.

- **EDU-based Context Compressor** (Zhou et al. 2025, arXiv:2512.14244) — structure-then-select compression via Elementary Discourse Unit decomposition. Orthogonal to this work in that the compression mechanism is discourse-structural rather than clustering-based.

This family shares with our work the framing of **”clustering-driven context compression”**. The fact that our **§1 motivation (the three axes of cognitive load / bandwidth / cost)** and Bohdal et al.’s **”balances context efficiency and personalization quality”** arrived at the same problem space independently means LayerForge is not an isolated design but **a position within a problem space that multiple research groups are working on in parallel in 2025-2026**.

1.2.4 Community detection method comparisons A family that systematically benchmarks / theoretically analyzes multiple community-detection methods:

- **Lancichinetti & Fortunato (2009)** — proposed the LFR synthetic benchmark, the standard evaluation infrastructure for community-detection algorithms.
- **Aldecoa & Marín (2013, Scientific Reports)** — systematically evaluated Newman modularity / CPM / RB / RN / MCL / Infomap, etc. on closed benchmarks, reporting that CPM **”produces accurate results only in the easy ends of the benchmark”** and **”shows some instability”**. Our §4-§5 small-N text results can be positioned as a contrast axis against these synthetic results.
- **Traag et al. (2011, 2019)** — the theoretical basis of CPM’s resolution-limit-free property (2011) and the Leiden refinement (2019).
- **From Leiden to Pleasure Island: The Constant Potts Model as a Hedonic Game** (Felipe, Avrachenkov, Menasche 2025, arXiv:2509.03834) — recasts CPM as a hedonic game (potential game), proves that local-utility optimization converges to equilibrium in pseudo-polynomial time, and introduces robust partition criteria. **Directly related to §5 of this work (CPM failure-mode analysis)**, but on synthetic / random graphs; our small-N text similarity graphs are complementary empirical observations.

These works center on synthetic graphs / closed benchmarks / theoretical analysis. To the best of our search, **a systematic Newman-vs-CPM comparison on small-N text similarity graphs (LayerForge’s operational range N=20-40)** does not exist elsewhere at the time of this work.

1.2.5 LayerForge positioning Against the above prior art, this study differentiates on the following five points:

1. **vs G2T (Zhang et al. 2023)**: Shares the substrate (sentence embedding + community detection on similarity graph), but this study focuses on the **systematic Newman-vs-CPM comparison in the small-N text regime + a three-failure-mode classification (§5)**, and additionally combines **SCA distillation (R3), HERCULES skeleton (R4), the 4 ± 1 cognitive constraint (R2), and a Mode A/B/C application layer**. A direct head-to-head comparison against G2T is out of scope for this paper (future work).
2. **vs Clustering-driven Memory Compression (Bohdal et al. 2026, Samsung)**: A concurrent prior art that fully matches the top-level framing of clustering-driven context compression. We differentiate on four axes — (i) **target**: LayerForge operates on corpus passages, Bohdal on user-specific memory tokens inside the LLM; (ii) **goal**: LayerForge converges AI output by concern (reducing cognitive load + bandwidth + cost on three axes), Bohdal fits the context budget while preserving personalization; (iii) **method**: LayerForge uses community detection on a similarity graph (Newman / CPM switchable), Bohdal uses similarity averaging + merge; (iv) **cognitive constraint**: LayerForge explicitly adopts Cowan (2010) 4 ± 1 as an ergonomic upper bound, while Bohdal only uses the external context-budget constraint. The two studies are **independent arrivals at the same problem space**, and this paper offers a complementary contribution on the corpus-passage side by empirically evaluating method choice (Newman vs CPM) inside the community-detection family.
3. **vs BERTopic / Top2Vec / CETopic**: This study takes **per-passage hard partitions** as the primary output and provides **deterministic Mode A/B/C** on top of the layer structure. BERTopic-family works take per-document topic distributions as the primary output, so the task formulation differs and they are not suitable as direct baselines (§2.1).

4. **vs RAPTOR / GraphRAG / Core-based GraphRAG / HERCULES**: This study **intentionally decouples LLM from the analysis core** and restricts LLM involvement to Boundary 1/2 (R5). This is a design choice aimed at **reproducibility of concern-wise convergence**, the opposite direction from the LLM-integrated family. With Core-based GraphRAG we share the recognition of "Leiden's reproducibility problem", but we diverge by going for **failure-mode documentation + method-selection signal inside the modularity family** rather than replacement (k-core).
5. **vs Aldecoa & Marín / Traag / Felipe et al. (Pleasure Island)**: Existing community-detection comparisons center on synthetic / random / closed benchmarks. To our knowledge, this paper is the first to empirically test "whether resolution-limit-free works" in the small-N text domain. The core contribution of §4 / §5 lies here.

In other words, LayerForge is **an application that places SCA + HERCULES + 4 ± 1 + Mode A/B/C on top of G2T's design space**, and the core contribution narrows to **(i) the implementation report of that combination, and (ii) the systematic Newman-vs-CPM comparison in the small-N text regime + the three-failure-mode classification (§5)**. We do not claim "community-detection method comparison itself" as a novel contribution; the paper should be read as **"empirical evaluation and documentation of known methods within this domain"**. The framing overlap with Bohdal et al. (Samsung 2026) does not negate this work's novelty; we record it as the fact that **two studies independently identified the same problem space**.

§2. Why these technologies — Necessity of the five-component combination

Extracting requirements from the §1 motivation:

Requirement	Technology choice
(R1) Extract "concern-wise units" from a corpus	Community detection on similarity graph
(R2) An ergonomic upper bound for the reader's cognition	4 ± 1 layers (Cowan's working memory capacity)
(R3) Extract the "core information" of each layer	Per-layer distillation (SCA-style)
(R4) Scalability to large corpora	Hierarchical KMeans foundation (HERCULES-style)
(R5) Determinism that "does not filter AI output with AI"	LLM-free analysis core, AI involvement only at bound

2.1 (R1) Why community detection

An established method for **unsupervised** automatic extraction of "related sentence groups". RAG extensions (RAPTOR, GraphRAG, HippoRAG) use Louvain/Leiden for passage clustering under the same spirit. This work belongs to that family, but differs by **decoupling community detection from LLM summary** under the R5 constraint (discussed later).

Methods that output document-level topic distributions (the topic-modeling family, e.g. LDA, BERTopic) are **not primarily aimed at producing passage-level hard partitions**, so they are not appropriate as direct baselines for this work's task formulation of "which layer does each passage belong to". The comparison target of this paper is restricted to **choice inside the community-detection method (Newman vs CPM)**.

2.2 (R2) Why 4 ± 1

Cowan (2010) estimates the central short-term-memory capacity at 3-5 chunks. If we require that "concern-wise convergence be easy for the reader", placing the **ergonomic upper bound** of layer count here is natural. This is a design decision to **adopt this as a target value**, not a claim that " 4 ± 1 is a universal cognitive constant".

2.3 (R3) Why SCA distillation

Community detection produces "which sentence belongs to which layer", but does not produce a **representative expression** for the layer. To hand a layer to a reader or downstream agent, a short form of "what

topic this layer is about” is needed. Eichin et al. (2024)’s Semantic Component Analysis distills per-cluster representative components, matching this requirement.

2.4 (R4) Why HERCULES

Flat 1-level community detection is sufficient for the main results of this work, but we adopt a **hierarchical KMeans skeleton** in advance, anticipating future operation such as ”recursively expanding an N=10K large corpus into $4 \times 4 \times 4 \times 4 = 256$ levels” (the current implementation defaults to `max_depth=1` and is recursively expandable via an option).

Note that HERCULES, at the time of this work, exists **only as a GitHub reference implementation (bandeerun/pyhercules, MIT) with no peer-reviewed publication**. We reference HERCULES as a ”prior OSS implementation that adopted hierarchical KMeans” for positioning, not as a citation base for its own theoretical superiority. The substantive basis for R4 is the design judgment that ”recursive KMeans functions as a skeleton for large-corpus extension”; HERCULES is one implementation example (= reference target) of that.

2.5 (R5) Why a deterministic core

As stated in §1, **the stochastic behavior of LLMs destroys the reproducibility of concern-wise convergence**. If different layerings emerge for the same input, the ”converged output” cannot be trusted. This work uses:

- **Input boundary:** natural language $\rightarrow$ structured (Boundary 1), LLM involvement allowed
- **Analysis core:** deterministic (community detection + KMeans + SCA are all deterministic algorithms)
- **Output boundary:** structured $\rightarrow$ natural language (Boundary 2), LLM involvement allowed (optional; in this work Modes A and C operate without AI)

Under this constraint, the analysis results guarantee **same input + same hyperparameters $\rightarrow$ same output**. Here, hyperparameters include the community-detection method choice (`community_method="newman"` | `"cpm"`), random seed, `target_range`, embedding model, etc. Reproducibility holds with these fixed; switching methods can yield different partitions (demonstrated in §4.3).

2.6 Summary — Necessity of the combination

The five technology choices are each derived from independent requirements (R1-R5), and **conversely, alternative choices that satisfy each requirement do not fit this study’s task**:

- Skip community detection $\rightarrow$ cannot automatically extract concern-wise units
- Drop $4 \pm 1 \rightarrow$ lose the basis for cognitive-load reduction
- Skip SCA $\rightarrow$ no representative expression per layer
- Drop HERCULES $\rightarrow$ no base for large-scale expansion
- Drop determinism $\rightarrow$ self-contradiction by trying to filter AI with AI for the §1 problem

§3 shows the **design rationale** for how this combination implements the §1 motivation (R1-R5). §4 empirically tests the validity of the **most important choice inside the combination (the community-detection method, Newman vs CPM)**. **Individual ablations of SCA distillation / HERCULES hierarchy / the 4 ± 1 constraint / determinism are out of scope** for this paper, and are stated in §7.5. The claim of this paper is restricted to ”an empirical method comparison on the most important choice inside the combination, showing that the Newman backend fits this domain”.

§3. Approach — LayerForge implementation

3.1 Pipeline

[input: list of passages, optional query]

(B1) `parse_to_structure` - natural language $\rightarrow$ FormulationInput (optional LLM, has mechanical fallback)

`build_similarity_matrix` - sentence-transformers cosine similarity

`find_valid_scale / find_cpm_resolution`
- find K in the 4 $\pm$ 1 range via threshold (Newman) or (CPM)

`detect_communities` - Newman spectral + KMeans / CPM-Louvain (community_method option)

`compute_modularity` - Q (both methods) + cpm_h (when CPM)

`distill_layer` - per-layer SCA (UMAP + HDBSCAN-style components)

`hierarchy_to_layer_summaries` - { layers: [...], inter_layer_relations: [...] }

(B2) `render_to_natural` - structured $\rightarrow$ natural language (optional LLM)

[output: 4 $\pm$ 1 layers + per-layer reps + optional natural-language rendering]

3.2 Three modes of operation

Mode	CLI	Purpose
A (decompose)	<code>layerforge-decompose</code>	Decompose a corpus into 4 $\pm$ 1 layers; pull only the layer(s) of interest from the corpus
B (decide)	<code>layerforge-decide</code>	Layer-organize decision information; track concern transitions via open/close
C (compress)	<code>layerforge-compress</code>	Compress an AI's verbose output into the layer subset relevant to the query (optional LLM)

3.3 Claude Code skill form

LayerForge runs standalone as a Python CLI, but is operationally placed as a **Claude Code skill**:

- **LayerForge itself holds no API key** (when LLM involvement is needed at Boundary 1/2, it is executed via Claude Code, where API keys are managed)
- A PostToolUse hook automatically validates the output schema
- `.claude/skills/layerforge/SKILL.md` functions as **an instruction sheet for the Claude using the skill**

In this form, LayerForge can be inserted into existing AI workflows as a "**peripheral tool of the LLM**", while **the analysis core preserves LLM-free determinism** (Boundary 1/2 allow optional LLM involvement as per §3.1, with mechanical fallback available).

3.4 Dual community-detection backend

The central engineering choice of this work is the community-detection method. The implementation provides both:

- **Newman (default)**: threshold + spectral algorithm + KMeans on embeddings; quality measured by modularity Q
- **CPM (opt-in)**: Leiden-CPM (self-implemented, MIT-pure); maximizes the H function with the (n_c choose 2) quadratic penalty

Switching is done via the `community_method` option:

```
layerforge_core(input, community_method="newman") # default
layerforge_core(input, community_method="cpm")    # opt-in
```

3.5 Reproducibility infrastructure

Artifact

scripts/k_sweep/correlation_data.py (119 rows)
scripts/k_sweep/heatmap_N_x_K.py (196 attempted cells = 7 N × 14 K × 2 methods, of which 167 succeeded — the remainder failed)
scripts/k_sweep/cpm_compare.py (95 rows + ARI/NMI)
scripts/k_sweep/multi_corpus_verify_v2.py (8 conditions)
scripts/k_sweep/k10_multi_corpus.py
scripts/k_sweep/run_robustness.py (32 setting-method aggregates, each holding 8 K-range measurements; 256 K-range measurements)
tests/integration/test_real_data_20ng.py
tests/axioms/test_cpm_karate_club.py

All raw CSV / JSON / PNG are bundled in git, pinned by commit hash.

§4. Empirical findings — Observations that held

This section is written as a report of "what was found to hold". We avoid claims such as "superior" or "best", and place **the measured numbers and their ranges** at the center.

4.0 Terminology for K (preliminaries for this section)

Term	Definition	Used in
<code>target_range</code>	the range of K to search (e.g., $4 \pm 1 = (3, 5)$)	argument of <code>find_valid_scale</code> / <code>find_valid_range</code>
<code>K_actual</code>	the number of communities obtained as a run result	all figures / tables
<code>K_optimal</code>	the K that maximizes Q in a K sweep (Newman)	§4.1, §4.4
<code>Q peak K</code>	synonym for <code>K_optimal</code>	§4.1
<code>n_themes</code>	the ground-truth theme count of the corpus	§4.4 (H_struct), §4.6 (20NG)
<code>K=10</code>	a specific operational candidate value (AI input compression)	§4.5
<code>default K</code>	the probe value in the method-selection rule (K=4 in this paper)	§5.7

§4.1 sweeps Q peak without pinning `target_range` to 4 ± 1 . §4.4 measures, for each setting, whether `K_actual` tracks the corresponding `n_themes`. The difference in corpus setups between the two is stated explicitly in §4.4.

4.1 N-dependent behavior of Q peak K (Newman)

The corpus in this sub-section is a **cross-domain mpnet corpus** (real-world markdown corpus of the KDF-perovskite project, four themes — philosophy / exploration / proof / blog, `n_themes = 4`), sampled at `per_theme = 2,3,4,5,6,8,10` to vary N (a different corpus family from the synthetic disjoint-vocabulary corpus of §4.4; see §4.4's reconciliation for the relation). The measurement question is "**does Q peak K**".

move when the sampling density of the same corpus structure varies?” — a check of whether the theoretical Q degeneracy of Good (2010) is reproduced in this implementation.

Q max over 7 N values $\times$ 14 K values $\times$ Newman backend:

N	Q peak K	Q peak value
8	3	0.403
12	6	0.778
16	11	0.720
20	6	0.716
24	9	0.646
32	4	0.550
40	6	0.610

Observation: For the same corpus structure, varying N (sample size) makes Q peak K fluctuate in the range 3-11. This demonstrates that the Newman Q degeneracy theoretically established by Good et al. (2010) is reproduced in this implementation. The independent contribution of this paper is not the discovery itself, but **the empirical dataset on the N $\times$ K axes**.

4.2 Method-agnostic monotonicity of the above-limit fraction

Using the Fortunato & Barthélemy (2007) resolution limit ($\sqrt{L/2}$) as a baseline, we compute the fraction of communities exceeding the limit. We present as the primary result the **common-reference measurement that directly supports the method-agnostic claim** (`scripts/k_sweep/data_current/cpm_compare_data.csv`, comparing both methods on the same graph thresholded by the corpus-wide median similarity):

N	K	Newman above-limit	CPM above-limit
12	3	0.67	0.67
12	4	0.25	0.25
20	4	0.50	0.50
24	4	0.75	0.75
24	5	0.60	0.60
40	5	0.80	0.60

Observation: In 5 of 6 cells the above-limit fraction matches **exactly** between the two methods; only N=40, K=5 diverges (Newman 0.80 / CPM 0.60). Both Newman and CPM are monotone-decreasing in K, and the two curves are nearly the same shape in this paper's operational range (N=12-40). The N=40 divergence is consistent with the regime §4.3 shows ("partitions become divergent at N 32") — on the same graph, different methods produce different partition shapes and hence different cluster-size distributions, which can move the above-limit fraction apart.

→ This work empirically shows (within the tested range) that **the above-limit fraction functions as a supplementary K-selection signal that does not depend on the community-detection method**. The asymmetric-reference version of the same cells, measured at the chosen by Newman's own `find_valid_scale`, is repeated in §5.2 and handled on the mechanism-analysis side (to separate the insights from the two reference choices).

4.3 Newman vs CPM partition agreement (ARI by N)

Running both methods on the same (N, K) cell and measuring the Adjusted Rand Index between the partitions:

N	ARI mean	ARI max
12	0.871	1.000
20	0.728	1.000
24	0.689	1.000
32	0.490	0.779
40	0.411	0.842

Observation: At N 24 the partitions nearly coincide (perfect match at some K), while at N 32 they diverge (at most 0.85). The naïve expectation of "Newman and CPM produce the same result" is **valid only on the small-N side** of this domain; in the operational range (N=20-40), method choice was found to affect partition structure.

4.4 H_struct (Q peak K = n_themes) tracking

This sub-section measures a **different axis** from §4.1:

- §4.1: "how does Q peak K move **when N varies for the same corpus structure**" (Q degeneracy reproduction)
- §4.4: "for **corpora with different n_themes**, does Q peak K match n_themes" (test of the H_struct hypothesis)

16 settings = 4 **n_themes** (3, 4, 5, 7) × 2 **embedders** (MiniLM, mpnet) × 2 **seeds** (42, 123). For each setting we sweep K ranges over {1-2, 2-3, 3-4, 3-5, 5-7, 6-8, 8-10, 10-12}. The two-tier design intent is: (a) continuously covering K=1 through 12 with overlap to confirm binary-search stability of `find_valid_scale` over the whole K span; (b) densely covering Cowan's 4 ± 1 (=3-5) with three ranges (2-3, 3-4, 3-5) to reduce error in this paper's region of interest (4 ± 1). For each range, we take the `K_actual` that maximizes Q and compare against the corresponding `n_themes`:

- Newman: 14/16 (**87.5%**) **exact match** (`K_actual == n_themes`)
- CPM: 3/16 (**19%**) **exact match**

Observation: The H_struct hypothesis "Q peak K tracks the corpus's natural theme count" is **strongly supported for the Newman backend in this domain of 16 settings (synthetic disjoint-vocabulary corpus)**, and not supported for the CPM backend. Since 16 settings is a small sample, we do not perform formal statistical significance tests (e.g., binomial test) in this paper, and qualitatively evaluate the effect size (87.5% vs 19%).

Scope limitation (important): The 87.5% tracking above is **a result on a synthetic disjoint-vocabulary corpus**, and this paper does **not** make the strong claim that "Newman's `find_valid_scale` pipeline automatically tracks `n_themes` on any corpus". On the external benchmark (20 Newsgroups, N=100, §4.6), even Newman returns K=3 under the default `target_range=(3,5)` and fails to track `n_themes=4`. This reflects the gap between the synthetic disjoint-vocabulary condition and public-corpus vocabulary overlap, and is positioned mechanistically in §5.4 (the `find_cpm_resolution`-style calibration design prefers the lower end of `target_range`). The H_struct claim of this section should be read as **partial support under the synthetic-corpus condition**.

→ **Figure:** `figures/new/fig_h_struct.png` — a 2-panel scatter (Newman | CPM) of `K_actual` vs `n_themes`, with a diagonal indicating perfect tracking. The Newman side concentrates near the diagonal, while the CPM side systematically falls below (visualizing under-merging).

Relation to §4.1 (reconciliation): §4.1 and §4.4 use different corpus families:

Item	§4.1
Corpus origin	real-world cross-domain (philosophy / exploration / proof / blog markdown of the KDF-perovs)
Axis varied	sampling density (per_theme) for the same corpus structure
Primary measurement	fluctuation of Q peak K with N

Item	§4.1
Result	Q peak K fluctuates in 3-11 with N (Q degeneracy reproduction)

The two measure **different questions on different corpus families** and do not contradict: §4.1 measures the "degenerate effect of sampling density inside the same corpus" on a real-world corpus, and §4.4 measures "tracking across corpora with different `n_themes`" on synthetic corpora. The real-world cross-domain corpus has more vocabulary overlap than §4.4's synthetic corpus, so it is expected behavior that Q peak K in §4.1 does not pin to `n_themes=4`.

4.5 K=10 self-routing (AI input compression candidate)

Definition of self-routing accuracy: for each passage `p`, treat the text of `p` itself as a query, compare the embedding of `p` against all layer centroids, and select the layer with the closest centroid. The fraction of cases in which the chosen layer matches `p`'s layer assignment. This is a **weaker test** than LLM-side paraphrased query routing, but sufficient to verify algorithm-level partition consistency. Production query routing is out of scope for this paper (see §7 Limitations).

8 conditions (2 corpora $\times$ 2 embedders $\times$ 2 methods) $\times$ K=10 cell:

condition	Newman self-routing	CPM self-routing
same-domain 5themes / MiniLM	30/30 (100%)	29/30 (97%)
cross-domain 4themes / MiniLM	24/24 (100%)	23/24 (96%)
same-domain 5themes / mpnet	30/30 (100%)	30/30 (100%)
cross-domain 4themes / mpnet	24/24 (100%)	24/24 (100%)

Observation: At K=10 self-routing stays in 96-100%, and the theoretical minimum read-fraction on the partition-consistency metric is $1/K = 10\%$ (a $10\times$ reduction). The two methods have similar accuracy. **On the self-routing metric, K=10 as an AI-input compression target holds method-agnostically.**

Important scope limitation: The " $10\times$ reduction" above is the **theoretical minimum value of partition consistency**, and **production retrieval accuracy under paraphrased queries is not measured in this section**. What self-routing measures is intra-cluster tightness — "does a passage, used as its own query, return to the layer it belongs to?" — and there is a gap between this and production query routing (which involves paraphrase and query-passage semantic gap). Reading these numbers as "achieving $10\times$ AI input compression" would be a leap; this section reports only that the **necessary condition for partition consistency** is satisfied. Verification of production query routing accuracy is out of scope for this paper (see §7.2 / §7.5).

Granularity dependence vs §4.4 / §4.6: The phenomenon that the method-difference size is very different between this section (K=10) and §4.4 (K = `n_themes`) is explained mechanistically in §5. In short: **in the K = `n_themes` regime, Newman picks up the true thematic structure via the Q peak while CPM under-merges**, so method differences become visible; meanwhile **in over-clustering regimes such as K=10, both methods are forced into "more clusters than themes"**, so partition fragmentation dominates over the ground-truth distinction and method differences become harder to observe. The granularity regime determines whether method differences surface.

4.6 20 Newsgroups (external benchmark)

Four thematically distinct newsgroups (sci.med / sci.space / rec.sport.hockey / talk.politics.guns) with 25 docs/topic = **N = 100 documents** (only in this sub-section, serving also as a scaling check from N=20-40 in §4.1-4.5).

Measured K: both methods are run with `target_range = (3, 5)` (Cowan's 4 ± 1 default range). In the current default behavior of the implementation, both methods return K=3 (`tests/integration/test_real_data_20ng.py`

admits Newman $\{3,4,5\}$ and CPM $\{2,3,4,5\}$). **The phenomenon that both methods plateau at $K=3$ is a symptom of the K -calibration design of this implementation (both `find_valid_scale` and `find_cpm_resolution` are specified to prefer the lower end of `target_range`; mechanism explained in §5.4), and the failure to reach `n_themes=4` is not an intrinsic limit of Newman.** The Newman ARI of 0.557 under forced $K=4$ shows the level reachable without going through calibration. The obtained partition is compared against the **ground truth (`n_themes = 4`)** and Adjusted Rand Index is measured (`sklearn.metrics.adjusted_rand_score`):

method	K_{actual} (default)	ARI vs ground truth
Newman	3	0.430
CPM	3	0.239
chance baseline (random partition)	—	0

For reference, fixing $K=4$ yields Newman ARI **0.557**, and $K=5$ yields 0.313. CPM stays at ARI 0.24 across $K=3,4,5$ (under-merging keeps it in the same partition family regardless of K).

→ **Figure:** `figures/new/fig_20ng_ari.png` — bar chart (Newman / CPM $\times K=3,4,5$), with the chance-baseline line and Newman's $K=4$ best (0.557) emphasized in annotations. The gap of Newman $1.8\times$ at default $K=3$ and Newman $2.3\times$ under forced $K=4$ is visually clear.

Observation: The external validity on a public benchmark is that **at the default setting, the Newman backend exceeds the CPM backend by about a factor of 1.8** (Newman 0.430 / CPM 0.239). Both methods clearly exceed the chance baseline, and Newman's advantage is observed stably. Newman's ARI=0.557 under forced $K=4$ shows that Newman can reach higher ARI at an appropriate K . CPM plateaus at 0.24 regardless of K , consistent with the (`n_c choose 2`) penalty effect (mode (c)) of §5.3.

§5. Why CPM Underperforms — Three failure modes and operational policy

The most counter-intuitive result observed in §4 is that **CPM is consistently outperformed by Newman**. Traag et al. (2011) mathematically established CPM's resolution-limit-freeness via the subgraph-invariance property, positioning it as a **correct fix** for Newman's Q degeneracy (Good 2010). Nevertheless, in this study's small- N text domain CPM degraded in the opposite direction. This section organizes the mechanism by separating it into **three distinct failure modes**. The opposite-direction behaviors of $K=4$ over-split on Karate Club (§7.3) and $K=3$ under-merge on 20 Newsgroups (§4.6) are positioned consistently as different modes.

5.1 The three failure modes

The causes of CPM-Louvain's deviation from the ground truth can be separated, within the verification range of this work, into **at least three independent mechanisms**:

Mode	Content
(a) Louvain refinement gap	There exist macro partitions unreachable by vanilla Louvain's single-node moves (a)
(b) Calibration bias	This implementation's <code>find_cpm_resolution</code> bisects and is designed to prefer th
(c) (<code>n_c choose 2</code>) penalty effect	In regimes where the resolution limit does not bind under small N , CPM's quadratic

Attribution of the numerical gaps in this paper:

- **§4.4 `H_struct` 87.5% vs 19%** (synthetic disjoint-vocabulary, K_{actual} taken at Q peak for each setting) → primarily (c). The K range densely covers around `n_themes`, so the influence of (b) is small.
- **§4.6 default $K=3$ (20NG)** → dominated by (b) calibration bias. Both methods converge to the lower end $K=3$ of `target_range=(3,5)`.

- **§4.6 Newman 0.557 vs CPM 0.24 under forced K=4** → (c) penalty effect. The residual CPM underperformance even after avoiding calibration.
- **§7.3 Karate K=4 over-split** → (a) Louvain refinement gap. Cannot reach the K=2 macro split regardless of .

5.2 Structural difference of the objective functions

Comparing the objective functions of the two methods:

- Newman: $Q = (1/2L) \sum_{ij} [A_{ij} - (k_i k_j / 2L)] (c_i, c_j)$
 - Internal edges compared against a null model (expectation $k_i k_j / 2L$)
 - No explicit penalty on cluster size
- CPM: $H = \sum_c [m_c - \frac{1}{2} \times (n_c \text{ choose } 2)]$
 - From intra-edges, **the cluster pair count** $\times$ is subtracted
 - Penalty grows **quadratically** in the cluster size n_c

→ **Figure:** `figures/new/fig_cpm_mechanism.png` — a comparison plot of penalty/null-model contribution as a function of cluster size n_c . The CPM ($n_c \text{ choose } 2$) quadratic curve exceeds Newman's linear null-model reference, with the "under-merging zone" shaded.

The **common-reference** measurement shown in §4.2 (both methods compared on the same graph thresholded at the corpus's median similarity) has above-limit fraction matching exactly in 5 of 6 cells in the operational range $N=12-40$. That is, in this domain **Newman's resolution limit is not operationally binding**, and the theoretical motivation for adopting CPM (limit-freeness) targets **a problem that is not a problem in this domain**. What remains is only the effect of CPM's own penalty structure — this is the starting observation for §5.3 onward.

For reference, we repeat the same cells under the **asymmetric reference** (Newman thresholded at the chosen by its own `find_valid_scale`, CPM thresholded at the median similarity), which complements §4.2. This is useful as a reference for directly inspecting "whether Newman's resolution limit is binding", but we use the common-reference of §4.2 as the ground for method comparison:

N	K	Newman above-limit (asym)	CPM above-limit (asym)
12	3	0.67	0.67
12	4	0.25	0.25
20	4	0.75	0.50
24	4	0.75	0.75
24	5	0.40	0.60
40	5	0.60	0.60

(The difference on the Newman side comes from the -selection path, not from the partition structure on the graph. Source: `scripts/k_sweep/data_current/heatmap_data.csv`.)

5.3 Failure mode (c) — ($n_c \text{ choose } 2$) penalty effect (small-N text domain)

In LayerForge's operational range ($N=20-40$ cross-domain text corpus):

- edge density is moderate (neither sparse nor dense)
- as the common-reference data of §4.2 shows, the above-limit fraction lies in 0.25-0.80, with most cells 0.5 at $N = 20$ → Newman's resolution limit is not operationally binding
- when CPM is applied, the ($n_c \text{ choose } 2$) term relatively dominates m_c , "large clusters are H-loss" becomes the dominant force, and CPM selects a **systematically coarse partition** (under-merging that remains even under K-matched comparison)

This is the root cause of the numerical gaps **§4.4 (H_{struct} 87.5% vs 19%)** and **§4.6 Newman 0.557 vs CPM 0.239 under forced K=4**. Mode (c) is the component that remains even when K is fixed, and is the **structural** primary cause of Newman's advantage in this domain.

5.4 Failure mode (b) — Calibration bias of find_cpm_resolution

This implementation's `find_cpm_resolution` (and the corresponding `find_valid_scale` on the Newman side) is designed to **prefer the smallest K within target_range**. Specifically, `cpm_backend.py::find_cpm_resolution` (L262-263):

```
if K_min <= k <= K_max:
    best = (mid, labels, h, k)
    hi = mid # prefer smaller (smaller K)
```

This design reflects the operational judgment "a coarser partition is easier to interpret (under the 4 ± 1 constraint)" (if the working-memory upper bound is an ergonomic upper bound, smaller K is more desirable). As a side effect, however, **tracking failure occurs on corpora where n_themes does not match target_range[0]**.

The phenomenon in §4.6 (20 Newsgroups, `n_themes=4`, `target_range=(3,5)`) that both methods converge to $K=3$ is a symptom of this mode. Under forced $K=4$ Newman rises to $ARI=0.557$ (avoiding mode (b) leaves only the effect of mode (c)), but if we look only at default behavior, Newman also fails to track `n_themes`.

The reason **Newman achieves 87.5% tracking in §4.4 (synthetic corpus, K ranges densely covering around n_themes)** is that the K-range design places `n_themes` near `target_range[0]`, so the influence of mode (b) is small. We note in this paper that for external corpora, mode (b) becomes visible under the default `target_range=(3,5)`.

5.5 Failure mode (a) — Louvain refinement gap (cross-ref to §7.3)

This implementation's CPM-Louvain is vanilla Louvain (greedy single-node moves) and does not include the Leiden refinement (Traag 2019). On dense small graphs such as Karate Club, the $K=2$ macro split is not a local optimum of single-node moves, and even lowering leaves it stuck at $K=4$ sub-clustering (a known phenomenon in the literature). §7.3 of this paper makes it explicit that the Karate Club result ($ARI\ 0.595$, $K=4$) falls under this mode.

For the main results in the small-scale text domain of this implementation (§4.4, §4.6), the graphs are not dense small graphs and mode (a) is not dominant. However, the **phenomenon that K comes out larger than the ground truth in the external comparison (Karate Club)** is explained by this mode. It is an algorithm-engineering constraint, independent of CPM's penalty structure itself.

5.6 Implication — "resolution-limit-free is universally better" is domain-dependent

CPM's theoretical advantage manifests in **domains where the resolution limit binds**. In LayerForge's small-N text domain, as the §4.2 common-reference data shows, limit-binding is rare in the first place, and CPM's "fix" targets **a problem that is not a problem in this domain**. At the same time, CPM's own penalty structure (mode (c)) and calibration design (mode (b)) produce **different problems**, and Newman consequently exceeds CPM consistently.

This is not a direct refutation of Traag et al. (2011)'s theoretical result (CPM's subgraph-invariance). It is the mechanism observation that **method selection must be made in light of the operating-domain characteristics (whether the limit binds, the typical relation between n_themes and target_range, the dense/sparse degree of the graph)**.

5.7 Empirical method-selection rule (sketch)

Given the mechanism analysis in §5.2-§5.5, **empirical signals** for choosing "Newman or CPM" at operation time can be constructed:

Signal	Recommended threshold
Newman Q at default K (K=4)	Q 0.3 → Newman
Above-limit fraction at default K	0.5 → Newman safe
Edge density at	> 0.5 → consider CPM
Corpus size N	> 1000 → consider CPM
Cross-method ARI at probe K	< 0.3 → flag uncertainty

Pseudocode for the decision rule is detailed in the method-selection rule section of the companion document (GitHub repository: https://github.com/ChaiCroquis/LayerForge/blob/main/docs/08_empirical_findings.md). **We have not implemented it as code:** in LayerForge's operational range (N=20-40, moderate edge density), the rule outputs "newman" in ~100% of cases, so the implementation cost outweighs the information value. The design is to be used as a foundation when extending to other domains (N=1000+, dense graphs).

§6. Application examples

LayerForge's Modes A/B/C have application areas corresponding to each of the three costs of §1 (cognitive load / bandwidth / AI cost):

6.1 Mode A — Reducing the reader's cognitive load

Decompose a corpus into 4 ± 1 layers, and expand only the "layer(s) of interest" from the UI / CLI. The reader can judge the content from the representative expression of the layer of interest, without reading the full corpus.

6.2 Mode B — Managing concern transitions in decision information

Layer-organize decision information under consideration, and attach `open/close/settle` states. By separating items under decision (open) from items already decided (settled), the reader can focus only on the **currently activated concern**.

6.3 Mode C — Multi-agent bandwidth reduction

Compress AI agent A's verbose output (e.g., a 15K-token comprehensive answer) into the layer subset relevant to agent B's query (decision-less subset guarantee — i.e., it functions only as an information subset, with no LLM-side summarization). In this study, `scripts/multiagent_demo/` achieves a 63% context reduction (B-axis null result: the downstream agent's output quality is preserved even after compression).

Relation to §4.5's $1/K=10\%$ (different measurements): The "K=10 with $1/K=10\%$ ($10\times$ reduction)" shown in §4.5 is the **theoretical minimum read-fraction on the partition-consistency metric** (the upper bound saying that, assuming self-routing of a passage-as-query succeeds, reading only the one layer it belongs to is sufficient); the unit of measurement is "number of layers read / total layers". The 63% reduction of Mode C in this section is a **token-count-based empirical value**, comparing agent A's full output and the subset that agent B actually reads, in token units, in the demo script. The two are **different measurements** (theoretical minimum vs. production-empirical) and cannot be directly compared. The former is the floor on a fine-grained partition at K=10; the latter is the measured reduction in a realistic Mode C workflow.

§7. Limitations

7.1 Scope of validation

- **The operational target is the N=20-40 small-N text domain.** As an external benchmark, we scale up to N=100 on 20 Newsgroups (§4.6), and the conclusion goes in the same direction (Newman advantage). **The ARI table in §4.3 is N=12-40 and §4.6 is N=100, so the intermediate N range (40-100) is not directly swept in this paper,** but since Newman's advantage is observed in the same direction at both endpoints, we infer the behavior in the interpolated range to be in the same direction (confirmation is future work). **N > 1000 and dense graphs are unverified.**
- **The external benchmark is only one corpus (20NG)** — reproduction on other public corpora is future work.
- **CPM-Louvain has no Leiden refinement** — corresponds to the known Karate Club K=4 sub-clustering phenomenon in the literature; specifically positioned in §7.3.

7.2 B-axis null results

Improvement in the LLM's own behavior when LayerForge is integrated into an AI workflow is outside the scope of this study; the three trials we ran (hallucination benchmark, multi-agent drift verification, context filter ablation) all gave **null results**. This means the claim "LayerForge makes AI smarter" is not defensible. The contribution of this study lies in **the pipeline itself that narrows AI output by concern and hands it to the reader / downstream agent**; we do not claim improvement in AI behavior.

7.3 Correctness gate for the self-implementation

A head-to-head numerical-accuracy comparison against the reference implementation of CPM-Louvain (GPL `leidenalg`) was not performed due to MIT-license-compatibility issues. As a substitute, we put in place the following **three-stage indirect gate**:

1. **Synthetic 3-block test** (`tests/axioms/test_cpm_backend.py::test_cpm_partition_separates_three_blocks`): K=3 is perfectly recovered on 3 blocks (intra=0.9, inter=0.05).
2. **Karate Club ARI = 0.595** vs the empirical 2-community ground truth (chance baseline = 0): on the public Zachary (1977) graph, because of over-splitting due to the absence of the Leiden refinement (the K=4 sub-clustering phenomenon mentioned in §7.1), directly inspecting the partition gives K=4 clusters. From the cluster structure we can verify that this is a **sub-refinement** of the ground-truth 2-community split, and ARI=0.595 is an intermediate value indicating **the agreement between this sub-refinement and the ground truth** (perfect match would give ARI=1.0, random would give 0). This suggests that adding the Leiden refinement leaves room to reach the K=2 macro split.
3. **Determinism cross-check**: both Newman and CPM are reproducible under the same seed.

This is not a strong proof of "CPM is implemented correctly" (the local-optimum limitation in the absence of the Leiden refinement is mentioned in §5.5 / §7.1). It is positioned as **evidence that the minimum correctness requirement is met as a prerequisite for the empirical comparison against Newman**.

7.4 No universality claim for 4±1

We adopt Cowan (2010)'s working-memory capacity as a design target of this study, but we do not claim that "4±1 universally emerges on any corpus". In our measurements, K_{optimal} fluctuates in 2-11 depending on the corpus (§4.1); we keep its positioning as an ergonomic upper bound.

7.5 Ablations declared out of scope

This paper restricts its scope to validating the **community-detection method choice (Newman vs CPM)**. The following ablations on other choices within the combination are not performed:

- The effect of **SCA distillation on/off** on partition / output quality
- The effect of **HERCULES hierarchy depth = 1** (this paper's default) vs **depth > 1**

- The difference in cognitive-load reduction between **the 4 ± 1 constraint vs. no constraint (arbitrary K_{optimal})**
- Output-drift measurement under **LLM involvement on/off at Boundary 1/2**

These would be needed to validate the LayerForge combination as a whole, more broadly than this paper does. They lie outside the claim range of this paper (validity of the most important choice inside the combination, the community-detection method) and are left as future work.

§8. Conclusions

For the problem stated in §1 — **converging AI output by concern to reduce cognitive load / bandwidth / cost** — we showed in §3 that the five-component combination of §2 is a **design that satisfies the task requirements R1-R5**, and in §4 we empirically tested the most important choice within the combination (the community-detection method).

Main observations (within this domain and within the verification range of this paper):

1. **The above-limit fraction (the Fortunato-Barthélemy ratio) functions as a supplementary K-selection signal that does not depend on the community-detection method (§4.2).**
2. **Within the test range (16 settings + 20 Newsgroups at $N=100$), the Newman backend exceeded the CPM backend by a large effect size (H_{struct} 87.5% vs 19%; 20NG ARI 0.430 vs 0.239 at default $K=3$; Newman reaches 0.557 when $K=4$ is forced; §4.4 / §4.6). Because of the small sample, we do not perform formal statistical significance tests and report qualitatively by effect size.**
3. **CPM's underperformance is mechanistically explainable:** the (n_c choose 2) quadratic penalty induces under-merging. The proposition "resolution-limit-free is universally better" does not hold domain-independently (§5).
4. **On the self-routing metric (a partition-consistency test that uses each passage as its own query), the theoretical minimum read-fraction of $1/K=10\%$ at $K=10$ holds for both methods (§4.5).** This is not a measure of production query-routing accuracy; it reports that the **necessary condition for partition consistency** is satisfied. Production paraphrased-query routing accuracy, and the relation to the 63% empirical token reduction reported for Mode C in §6.3, are stated in §4.5 / §6.3; production accuracy verification is future work (§7.2 / §7.5).
5. **A method-selection rule can be constructed from empirical signals (§5.7; details in the companion document on the GitHub repository: https://github.com/ChaiCroquis/LayerForge/blob/main/docs/08_empirical_findings.md).** In this domain the rule outputs "newman" almost always, so we do not implement it at runtime and preserve it only in the docs.

LayerForge is released under the MIT license, can be operated as a Claude Code skill, and bundles reproducibility scripts and CSV / JSON. The numbers measured in this paper are pinned to the publication-time commit hash (see `git log`).

References

Foundational

- Cowan, N. (2010). The magical mystery four: How is working memory capacity limited, and why? *Current Directions in Psychological Science*, 19(1), 51-57.
- Zachary, W. W. (1977). An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4), 452-473.

Community detection theory and benchmarks

- Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS*, 103(23), 8577-8582.
- Fortunato, S., & Barthélemy, M. (2007). Resolution limit in community detection. *PNAS*, 104(1), 36-41.
- Lancichinetti, A., & Fortunato, S. (2009). Benchmarks for testing community detection algorithms on directed and weighted graphs with overlapping communities. *Physical Review E*, 80, 016118.
- Good, B. H., de Montjoye, Y.-A., & Clauset, A. (2010). Performance of modularity maximization in practical contexts. *Physical Review E*, 81, 046106.
- Traag, V. A., Van Dooren, P., & Nesterov, Y. (2011). Narrow scope for resolution-limit-free community detection. *Physical Review E*, 84, 016114.
- Aldecoa, R., & Marín, I. (2013). Exploring the limits of community detection strategies in complex networks. *Scientific Reports*, 3, 2216.
- Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9, 5233.
- Felipe, L. L., Avrachenkov, K., & Menasche, D. S. (2025). From Leiden to Pleasure Island: The Constant Potts Model for community detection as a hedonic game. arXiv:2509.03834.

Clustering-based topic modeling and graph-based text clustering

- Angelov, D. (2020). Top2Vec: Distributed representations of topics. arXiv:2008.09470.
- Rao, R. N., & Chakraborty, M. (2021). Vec2GC – A graph based clustering method for text representations. arXiv:2104.09439.
- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05794.
- Zhang, Z., Fang, M., Chen, L., & Namazi-Rad, M.-R. (2022). Is neural topic modelling better than clustering? An empirical study on clustering with contextual embeddings for topics. *Proceedings of NAACL*. (CETopic)
- Zhang, L., Liu, J., & Yan, Q. (2023). Graph2Topic: An opensource topic modeling framework based on sentence embedding and community detection. arXiv:2304.06653. (G2T)
- Eichin, F., Schuster, M., Groh, G., & Hedderich, M. A. (2024). Semantic Component Analysis: Discovering patterns in short texts beyond topics. arXiv:2410.21054.
- Janssens, W., Bogaert, M., & Van den Poel, D. (2025). LLM-assisted topic reduction for BERTopic on social media data. arXiv:2509.19365.

Hierarchical clustering, RAG and GraphRAG

- Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *ICLR*. arXiv:2401.18059.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A Graph RAG approach to query-focused summarization. arXiv:2404.16130.
- Xiao, Y., Dong, J., Zhou, C., Dong, S., Zhang, Q.-w., Yin, D., Sun, X., & Huang, X. (2025). GraphRAG-Bench: Challenging domain-specific reasoning for evaluating graph retrieval-augmented generation. arXiv:2506.02404.
- Han, H., Ma, L., Wang, Y., Shomer, H., Lei, Y., Qi, Z., Guo, K., Hua, Z., Long, B., Liu, H., Aggarwal, C. C., & Tang, J. (2026). RAG vs. GraphRAG: A systematic evaluation and key insights. arXiv:2502.11371.
- Hossain, J., & Saryüce, A. E. (2026). Core-based hierarchies for efficient GraphRAG. arXiv:2603.05207.

Context and memory compression

- Bohdal, O., Saha, P., Michieli, U., Ozay, M., & Ceritli, T. (2026). Clustering-driven memory compression for on-device large language models. arXiv:2601.17443.
- Zhou, Y., Lei, Y., Si, S., Sun, Q., Wang, W., Wu, Y., Wen, H., Chen, G., Qi, F., & Sun, M. (2025). From context to EDUs: Faithful and structured context compression via Elementary Discourse Unit

decomposition. arXiv:2512.14244.

Reference implementations

- `mainlp/semantic_components` (MIT, 2024) — SCA reference implementation by F. Eichin
- `bandeerun/pyhercules` (MIT, 2025) — HERCULES Python implementation (no peer-reviewed publication at the time of this work)

Project artifact

- LayerForge: <https://github.com/ChaiCroquis/LayerForge> (MIT licensed)
- Commit hash for measurements in this paper: see `git log` at submission time
- All CSVs, JSONs, PNGs in `scripts/k_sweep/` are reproducible from the corresponding `*.py` scripts.

Appendix A — Related work summary table

A reference table for the literature detailed in §1.2 (Related work and positioning). This appendix supplements §1.2; for the detailed positioning of each work, see §1.2.

Group (§1.2 ID)	Main works
Clustering-based topic modeling and graph-based text clustering (§1.2.1)	BERTopic (Grootendorst 2022), Top2Vec (Angeli et al. 2022)
Hierarchical RAG and GraphRAG (§1.2.2)	RAPTOR (Sarathi et al. 2024), GraphRAG (Edgar et al. 2024)
Context and memory compression (§1.2.3)	Clustering-driven Memory Compression (Lancichinetti & Fortunato 2009)
Community detection method comparison (§1.2.4)	Lancichinetti & Fortunato (2009), Aldecoa & Malmgren (2024)

LayerForge's contribution narrows to **(i) an implementation report of combining known techniques (G2T-like substrate + SCA + HERCULES + 4±1 + Mode A/B/C), and (ii) a systematic Newman-vs-CPM comparison in the small-N text domain + a three-failure-mode classification.** We do not claim **the community-detection method comparison itself** as novelty; the paper is positioned as **a report that contributes empirical data in this domain to the literature space.** The concurrent framing overlap with Bohdal et al. (Samsung 2026) does not negate the novelty of this work; we record it as the fact that two studies independently identified the same problem space.

Appendix B — Sensitivity analyses summary

A summary of the main outcomes of the sensitivity analyses detailed from §2 onward of the companion document on the GitHub repository (https://github.com/ChaiCroquis/LayerForge/blob/main/docs/08_empirical_findings.md):

- Embedding model (MiniLM vs mpnet): mpnet gives a cleaner Q peak; both methods go in the same direction.
- Random seed (42 / 123): K_optimal is stable; Newman's Q peak K is seed-invariant.
- Edge floor: above-limit fraction is monotonic in the same way under 0 (default) and median similarity (the CPM reference).
- N (corpus size): K=n_themes tracking is stable for Newman from N 20; CPM is weak and N-insensitive.

Document ends.